

TelemetriaMqtt v2 — Firmware

Como funciona, o que cada parte faz e em que estado está

Projeto de Telemetria FSAE

18 de setembro de 2026

Abstract

Este documento explica o firmware do ESP32 do projeto: qual é o seu papel no sistema, como o código está organizado, o que acontece do momento em que um quadro CAN chega ao fio até o valor de um sinal ficar disponível na memória, e como se testa cada parte. O firmware está **parcialmente implementado**: leitura e decodificação do CAN existem e são testadas (checkpoint 8); WiFi, MQTT e TLS ainda não (checkpoint 9). O documento separa com clareza o que já existe do que ainda é plano.

Contents

1	Papel do firmware no sistema	1
1.1	Por que um transceiver externo	1
2	Estado atual: o que existe e o que falta	2
3	Estrutura do projeto	2
3.1	Dois ambientes de compilação	2
4	Como o firmware funciona	3
4.1	Visão do fluxo de dados	3
4.2	Inicialização: <code>setup()</code>	3
4.3	Laço principal: <code>loop()</code>	3
5	O protocolo CAN que o firmware entende	4
5.1	Exemplo: um quadro vira dois sinais	4
5.2	Frequências diferentes, um único estado	4
6	O módulo <code>can_parser</code> em detalhe	4
6.1	Os dois tipos de dados	4
6.2	O que <code>parse_frame</code> faz, passo a passo	5
6.3	Leitura dos bytes	5
7	A camada TWAI: <code>can_twai</code>	6
7.1	Decisões desta camada	6
8	Como compilar, rodar e testar	6
8.1	O que os testes cobrem	7
8.2	O que os testes <i>não</i> cobrem	7
9	Limitações e pontos de atenção	7

10 O que vem a seguir	8
10.1 Checkpoint 9: WiFi, MQTT e TLS	8
10.2 Depois	8
11 Glossário rápido	8

1 Papel do firmware no sistema

O carro tem um módulo de distribuição de energia, o **PDM32** (AIM), que já mede as correntes das cargas, a tensão da bateria e outros sinais do motor. Ele transmite esses valores continuamente no barramento **CAN2** a 500 kbps, em 10 quadros diferentes (IDs 0x100 a 0x131) configurados no software Race Studio.

O firmware roda num **ESP32** e tem uma única missão: *ouvir esse barramento, entender os quadros e transformá-los nos 20 sinais de telemetria*, para que sejam enviados por MQTT ao resto do sistema (broker, ingestão, InfluxDB, Grafana). O ESP32 não mede nada por conta própria e não comanda nada no carro; ele só observa e repassa.



Figure 1: Cadeia física. As caixas tracejadas ainda não existem.

1.1 Por que um transceiver externo

O ESP32 tem o *controlador* CAN embutido (chamado **TWAI**, *Two-Wire Automotive Interface*, compatível com CAN 2.0), mas não tem o *transceiver*, o circuito que converte os níveis lógicos de 3,3 V nos níveis diferenciais do barramento. Por isso existe o SN65HVD230, que opera a 3,3 V e liga direto nos GPIOs do ESP32, sem conversor de nível.

2 Estado atual: o que existe e o que falta

Função	Situação	Checkpoint
Ler quadros do CAN2 pelo TWAI (500 kbps)	Implementado, compila; <i>não testado com hardware</i>	8
Decodificar os 10 payloads nos 20 sinais	Implementado e testado (20 testes nativos)	8
Imprimir resumo na serial para conferência	Implementado	8
Conectar ao WiFi	Não implementado	9
Montar o JSON e publicar por MQTT	Não implementado	9
TLS com certificado do broker	Não implementado	9
Validação com hardware e com o carro	Depende de o hardware chegar	11

Em termos práticos: hoje o firmware, ligado a um barramento CAN real, faria o ESP32 receber e decodificar os quadros e mostrar na serial um resumo a cada segundo. Nada sai do ESP32 pela rede. O arquivo `include/secrets.h.example` (credenciais de WiFi e MQTT) já existe como modelo, mas nenhum código o usa ainda.

3 Estrutura do projeto

Listing 1: Árvore de firmware/ (sem arquivos gerados)

```
1 firmware/
2   platformio.ini           dois ambientes: esp32dev e native
3   include/
4     secrets.h.example      modelo de credenciais (ainda não usado)
5   lib/can_parser/
6     can_parser.h           CanFrame, Telemetry, parse_frame()
7     can_parser.cpp         decodificação (C++17 puro)
8   src/
9     main.cpp               setup() / loop() do ESP32
10    can_twai.h / can_twai.cpp driver TWAI (só esp32dev)
11  test/
12    test_scaffolding/      prova de que o harness de testes roda
13    test_can_parser/       20 testes do parser
```

3.1 Dois ambientes de compilação

O `platformio.ini` define dois ambientes com propósitos diferentes:

Ambiente	Plataforma	Para que serve
esp32dev	ESP32 (framework Arduino)	Gera o firmware de verdade. Compila <code>src/</code> e <code>lib/</code> . Gravar exige o ESP32 conectado por USB.
native	PC (Linux)	Roda os testes unitários com o framework Unity, sem hardware. Compila só <code>lib/</code> e <code>test/</code> .

A divisão do código segue essa separação. Tudo o que tem *lógica* (a decodificação) fica em `lib/can_parser`, sem nenhuma dependência de Arduino ou ESP-IDF, e por isso compila nos dois ambientes. Tudo o que depende do chip (o driver TWAI, `Serial`, `millis()`) fica em `src/`, que só existe no `esp32dev`.

4 Como o firmware funciona

4.1 Visão do fluxo de dados

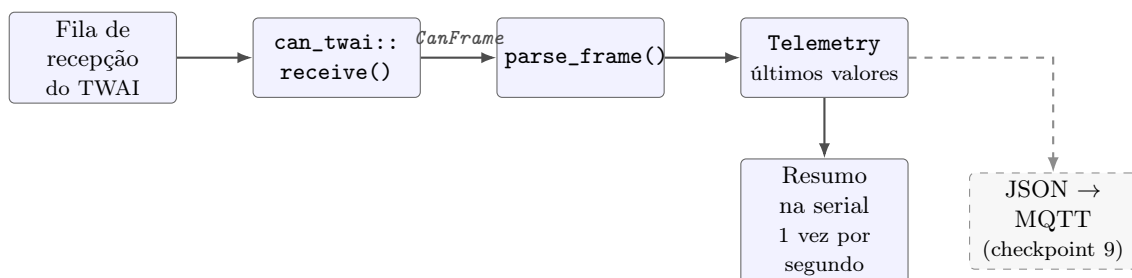


Figure 2: Caminho de um quadro dentro do firmware.

Os quatro blocos de cima acontecem o tempo todo, dentro do `loop()`. A leitura para a serial acontece uma vez por segundo. O envio por MQTT é o que falta.

4.2 Inicialização: `setup()`

`setup()` roda uma vez ao ligar o ESP32 e faz só duas coisas:

1. Abre a serial a 115 200 baud e imprime o nome do firmware.

2. Chama `can_twai::begin()`, que instala e liga o driver CAN. Se falhar, imprime TWAI: falha ao iniciar o driver CAN.

Se o driver não subir, o firmware **não para**: o `loop()` continua rodando e a linha de resumo aparece com os contadores em zero. A única pista é a mensagem de erro impressa uma vez no início.

4.3 Laço principal: `loop()`

A cada volta (sem delay, o laço gira o mais rápido possível), o `loop()` faz:

1. **Esvazia a fila de recepção.** Enquanto `can_twai::receive()` devolver um quadro, ele é entregue a `parse_frame()`. Se o parser aceitar, incrementa `frames_ok`; senão, `frames_ignored`.
2. **Confere o relógio.** Se passou 1 segundo desde o último relatório, imprime uma linha na serial.

Listing 2: O coração do `main.cpp`

```
1 void loop() {
2     can_parser::CanFrame frame;
3     while (can_twai::receive(frame)) {
4         if (can_parser::parse_frame(frame, telemetry)) {
5             frames_ok++;
6         } else {
7             frames_ignored++;
8         }
9     }
10
11     const uint32_t now = millis();
12     if (now - last_report_ms >= 1000) {
13         last_report_ms = now;
14         Serial.printf("CAN ok=%lu ignorados=%lu | rpm=%.0f ...\n", ...);
15     }
16 }
```

A linha de resumo (formato ilustrativo, não é saída medida) tem este aspecto:

Listing 3: Exemplo do formato da linha na serial

```
1 CAN ok=1980 ignorados=0 | rpm=6500 engineTemp=88.5 extBattery=12.60
```

Como ler: `ok` deve crescer sempre que o PDM estiver transmitindo (o PDM manda cerca de 99 quadros por segundo no total); `ignorados` deve ficar perto de zero; os três valores acompanham o carro. Diferença de tempo com `now - last_report_ms` usa aritmética sem sinal, então continua correta mesmo quando o contador de milissegundos dá a volta (após cerca de 49 dias ligado).

5 O protocolo CAN que o firmware entende

O contrato é a tabela “Plano de payloads” do `SCOPE.md`, que espelha o que foi configurado no Race Studio. Todos os valores são **little endian**; os `float` são IEEE 754 de 32 bits.

ID	Freq	DLC	Bytes 0–3	Bytes 4–7
0x100	20 Hz	8	rpm	throttlePosition
0x101	20 Hz	8	lambda	map
0x110	10 Hz	8	gpsSpeed	shifterCurrent
0x111	10 Hz	8	bicosD1Current	bicosD2Current
0x112	10 Hz	8	bobinasCurrent	poTotCurrent
0x113	10 Hz	8	poTotCrntAll	partidaCurrent
0x114	10 Hz	4	extBattery	—
0x120	5 Hz	8	ventoinhaCurrent	bombaCurrent
0x130	2 Hz	8	engineTemp	airTemp
0x131	2 Hz	8	brakeLightCurrent	bombaInput, giratoriaInput (uint16, bytes 4–5 e 6–7)

5.1 Exemplo: um quadro vira dois sinais

Suponha que chega um quadro com ID 0x100, DLC 8 e estes 8 bytes de dados:

00	50	9A	44	00	00	48	C1
bytes 0–3				bytes 4–7			

O parser separa em duas palavras de 4 bytes e reordena cada uma do little endian para o valor de 32 bits:

- bytes 00 50 9A 44 → 0x449A5000 → **1234,5** → rpm
- bytes 00 00 48 C1 → 0xC1480000 → **−12,5** → throttlePosition

(Os valores do exemplo foram escolhidos para a conta ficar fácil de acompanhar; não são leituras reais do carro.)

5.2 Frequências diferentes, um único estado

Os payloads chegam em ritmos distintos (20, 10, 5 e 2 Hz). Por isso o firmware não guarda “o último quadro”, e sim os **últimos valores conhecidos de cada sinal**. Cada quadro atualiza apenas os campos do seu ID e deixa os outros como estavam. A estrutura que faz isso é a Telemetry.

6 O módulo can_parser em detalhe

6.1 Os dois tipos de dados

Listing 4: firmware/lib/can_parser/can_parser.h (resumo)

```

1  struct CanFrame {                // um quadro CAN, já tirado do driver
2      uint32_t id;
3      uint8_t dlc;                 // quantidade de bytes de dados
4      bool    extended;           // true = ID de 29 bits
5      uint8_t data[8];
6  };
7
8  struct Telemetry {               // últimos valores dos 20 sinais
9      float rpm, throttlePosition, lambda, map;           // 0x100, 0x101
10     float gpsSpeed, shifterCurrent, ...;               // 0x110..0x114
11     ...                                                // 18 floats no total
12     uint16_t bombaInput;
13     uint16_t giratoriaInput;
14 };
15
16 bool parse_frame(const CanFrame &frame, Telemetry &out);

```

Duas propriedades de projeto importam:

- **Os campos de Telemetry têm os mesmos nomes das chaves do JSON** que o `mock_publisher` (o simulador em Python) já publica. Quando o checkpoint 9 serializar a estrutura, não precisará de tabela de tradução.
- **Sem *padding*.** São 18 float e 2 uint16, 76 bytes exatos. Isso permite aos testes comparar duas instâncias byte a byte; um `static_assert` nos testes garante a propriedade.

6.2 O que `parse_frame` faz, passo a passo

1. **Rejeita quadros estendidos** (ID de 29 bits): o PDM foi configurado com IDs de 11 bits, e um estendido com o mesmo número seria outro endereço.
2. **Consulta o DLC exigido pelo ID.** Uma função interna devolve 8 para quase todos, 4 para 0x114 e 0 para IDs fora da tabela. ID desconhecido ou DLC menor que o exigido: rejeita.
3. **Só então decodifica**, com um switch por ID, gravando nos campos correspondentes.
4. Devolve `true`.

A ordem é intencional: **toda a validação acontece antes de qualquer escrita**. Nenhum caminho devolve `false` depois de ter alterado um campo, então uma falha nunca deixa a telemetria meio atualizada.

Situação do quadro	Motivo	Resultado
ID fora da tabela	Não é um payload do PDM configurado.	Rejeita
Estendido (29 bits)	O PDM usa 11 bits.	Rejeita
DLC menor que o do contrato	Faltam bytes para ler os campos.	Rejeita
DLC maior que o do contrato	Os bytes do contrato continuam no mesmo lugar.	Aceita, ignora o resto

O protocolo não diz o que fazer com quadros fora do padrão; essa política é uma decisão de projeto, registrada no `SCOPE.md` e aberta a revisão.

6.3 Leitura dos bytes

Listing 5: Leitura de um float little endian

```
1 float read_f32_le(const uint8_t *p) {
2     const uint32_t bits = static_cast<uint32_t>(p[0])
3                           | (static_cast<uint32_t>(p[1]) << 8)
4                           | (static_cast<uint32_t>(p[2]) << 16)
5                           | (static_cast<uint32_t>(p[3]) << 24);
6     float value;
7     std::memcpy(&value, &bits, sizeof(value));
8     return value;
9 }
```

Os quatro bytes são montados em um inteiro e só então reinterpretados como `float` com `memcpy`, em vez de converter o ponteiro para `float*`. O resultado fica correto em qualquer endianness e sem risco de leitura desalinhada. Os `uint16` seguem a mesma ideia, com dois bytes.

7 A camada TWAI: `can_twai`

São 49 linhas, sem lógica de decodificação. Expõe duas funções:

Função	O que faz
<code>begin()</code>	Configura o driver (500 kbps, filtro aceita tudo, pinos TX/RX, modo NORMAL), instala e inicia. Devolve false se algum passo falhar.
<code>receive(frame)</code>	Tenta pegar um quadro da fila <i>sem bloquear</i> (timeout zero). Ignora quadros de requisição remota (sem dados). Copia id, DLC, flag de estendido e bytes para o <code>CanFrame</code> . Devolve false se a fila está vazia.

7.1 Decisões desta camada

- **Modo NORMAL, não LISTEN_ONLY.** O `SCOPE.md` registra que nada mais usa o CAN2. Se o ESP32 for o único receptor, ninguém daria *ACK* aos quadros do PDM; ele enxergaria erro em todo quadro e ficaria retransmitindo. No modo normal o controlador dá o ACK. O firmware nunca chama `twai_transmit()`, então não escreve dados no barramento. *A confirmar com hardware:* se existir outro nó dando ACK, LISTEN_ONLY passa a ser a opção mais segura.
- **Pinos: TX = GPIO21, RX = GPIO22.** São os do exemplo oficial do ESP-IDF, fora dos pinos de boot. *Provisórios:* conferir com a fiação real. Só duas constantes mudam.
- **Filtro aceita tudo.** Quem descarta o que não interessa é o parser, que é testado.
- **Leitura sem bloqueio.** Se a fila estiver vazia, `loop()` segue adiante em vez de esperar.

8 Como compilar, rodar e testar

Listing 6: Comandos (a partir de firmware/)

```

1 pio test -e native                # todos os testes (sem hardware)
2 pio test -e native -f test_can_parser # só a suíte do parser
3 pio run -e esp32dev                # compila para o ESP32
4 pio run -e esp32dev -t upload       # grava (ESP32 conectado por USB)
5 pio device monitor                  # abre a serial a 115200

```

8.1 O que os testes cobrem

Há 21 testes nativos: 1 de sanidade do harness e 20 do parser. Os 20 do parser provam:

- **Cada um dos 10 payloads** decodifica seus campos com valores conhecidos (inclusive um número negativo e o quadro curto 0x114).
- **A ordem dos bytes** dos `uint16` é little endian (usa valores como 0x0100, que distinguem a ordem; 0 e 1 sozinhos não distinguiriam).
- **Composição:** um quadro só mexe nos seus campos; os 10 juntos preenchem os 20 sinais; um quadro novo sobrescreve o valor anterior.
- **Entradas inválidas:** ID desconhecido, DLC curto (para os 10 IDs), DLC zero, DLC maior e quadro estendido. Nos rejeitados, o teste compara **Telemetry** byte a byte com o estado anterior para provar que nada mudou.

Os bytes esperados foram calculados de forma independente do parser (com `struct.pack` do Python) e escritos como literais, para que um erro de offset não passasse despercebido nos dois lados. O checkpoint 8 ainda validou os testes por *mutação*: o parser foi quebrado de propósito de três formas e a suíte detectou todas.

8.2 O que os testes *não* cobrem

`can_twai.cpp` e `main.cpp` só têm a garantia de **compilar**. Que o controlador realmente recebe quadros do PDM, que os pinos estão certos e que o ACK funciona só se comprova com o ESP32, o transceiver e uma fonte de quadros (o PDM ou outro nó). O CI também não compila o alvo `esp32dev`; roda só os testes nativos.

9 Limitações e pontos de atenção

Itens que não são defeitos do que existe, mas que valem ser conhecidos antes de seguir para o checkpoint 9.

1. **Fila de recepção pequena.** O driver foi configurado com `TWAI_GENERAL_CONFIG_DEFAULT`, cuja fila de recepção guarda **5 quadros** (conferido no cabeçalho do ESP-IDF instalado). Hoje o `loop()` não faz nada além de esvaziar a fila e imprimir uma linha por segundo, então gira muito mais rápido que a chegada dos quadros (não medido) e não há risco aparente. Mas WiFi, reconexão e publicação MQTT podem tornar o laço lento, e com cerca de 99 quadros por segundo a fila transbordaria e quadros seriam perdidos em silêncio. No checkpoint 9 isso deve ser tratado: aumentar `rx_queue_len` e/ou separar a leitura do CAN da rede em tarefas distintas.
2. **Valor “velho” fica congelado.** Telemetry guarda o último valor de cada sinal, mas não a idade dele. Se o PDM parar de transmitir, os valores ficam parados sem nenhuma indicação de que estão obsoletos.
3. **Zero antes do primeiro quadro.** Um sinal que ainda não chegou vale 0, indistinguível de um 0 real. O firmware não sabe hoje dizer “ainda não recebi”. Relevante para o JSON, que hoje contém sempre os 20 campos.
4. **Sem recuperação de erros do barramento.** Não há tratamento de *bus-off* nem consulta ao estado do driver (nenhum alerta do TWAI é habilitado). Se o controlador entrar em bus-off, ele para de receber e o firmware não percebe nem reinicia.
5. **Falha na inicialização não é fatal.** Como visto em `setup()`, o firmware segue rodando sem CAN.

Nenhum desses pontos foi tratado neste momento por estarem fora do escopo do checkpoint 8; ficam registrados aqui como insumo para o próximo.

10 O que vem a seguir

10.1 Checkpoint 9: WiFi, MQTT e TLS

Vai completar a linha tracejada das figuras. O plano, conforme o `SCOPE.md`:

- Ler credenciais de `secrets.h` (copiado de `secrets.h.example`, ignorado pelo git), nunca escritas no código.
- Conectar ao WiFi e ao broker com **TLS validando o certificado real** (Let’s Encrypt), sem `setInsecure()`.
- Publicar no tópico `telemetria/esp32/data` um JSON com os 20 sinais e um `ts` em milissegundos (o `millis()` do ESP32), o **mesmo contrato** do simulador em Python. Como a Telemetry já usa os nomes das chaves, a serialização sai direto dela.
- Usar o usuário MQTT `esp32_telemetria`, que a ACL do broker só deixa *publicar* nesse tópico, nunca assinar nem administrar.

10.2 Depois

Checkpoint 10: Traefik, Let's Encrypt e deploy na VPS. Checkpoint 11: validação ponta a ponta, que é quando o hardware entra em cena e as pendências de pinos e modo do TWAI se confirmam ou se corrigem.

11 Glossário rápido

Termo	Significado
CAN	Barramento serial usado em carros; vários módulos compartilham o mesmo par de fios.
Quadro (<i>frame</i>)	Uma mensagem CAN: um ID e até 8 bytes de dados.
ID	Identificador do quadro; aqui, 11 bits (0x100 a 0x131).
DLC	<i>Data Length Code</i> : quantos bytes de dados o quadro carrega (0 a 8).
TWAI	Nome do controlador CAN do ESP32 (<i>Two-Wire Automotive Interface</i>).
Transceiver	Circuito que adapta os sinais do controlador ao barramento físico (SN65HVD230).
ACK	Confirmação que um receptor dá no barramento para cada quadro recebido.
Little endian	Ordem de bytes em que o menos significativo vem primeiro.
PDM32	Módulo AIM que distribui a energia do carro e transmite os sinais.
Race Studio	Software da AIM em que se configuram os payloads do PDM.
Unity	Framework de testes unitários para C/C++ usado no ambiente native .